

NB05: Syntactic Analysis

Language Processing

LEI/2025-26

Nuno Macedo

Universidade do Minho

✓ 5.1 Syntactic Analysis

- **Syntactic analysis** is the second stage of the language-processing pipeline
- Unlike lexical analysis, which works over characters, syntactic analysis works over the *tokens* produced by the lexer
- A syntactic analyzer introduces and validates the structure of the language being processed
 - Its output is typically an **abstract syntax tree** (AST)
- **Context-free grammars** (CFGs) are commonly used to specify the syntax of computer languages
 - The alphabet of the grammar is usually the set of tokens produced by the lexer
 - A *parse tree* reflects the grammar directly, while the AST omits irrelevant syntactic detail
- This means that lexical analysis and syntactic analysis are interdependent
 - Decisions made at the lexer level directly affect the parser
- Although there are algorithms for parsing arbitrary CFGs, in practice we often restrict ourselves to smaller grammar classes for efficiency and simplicity



✓ Pipeline example: A language for lists

- Consider a simple language of lists of integers:
 - `[10, 20, 12]`
 - `[10]`
 - `[]`
- One possible CFG works directly on characters. Let:
 - $\Sigma = \{ \texttt{[}, \texttt{]}, \texttt{,}, \texttt{0}, \texttt{1}, \texttt{.}, \texttt{9} \}$
 - $N = \{ \texttt{List}, \texttt{Elements}, \texttt{Int}, \texttt{Digit} \}$
 - $S_0 = \texttt{List}$

```

List    → [ Elements ] | [ ]
Elements → Int | Int , Elements
Int      → Digit | Digit Int
Digit    → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

```

- This grammar generates the intended language, but it places unnecessary burden on the parser, which must inspect every character of the input
- In practice, we usually let the lexer recognize integer literals first, and then let the parser work on a stream of tokens
- To distinguish literal tokens from nonterminals, we write those tokens in quotes
 - This is similar to the notation used in BNF (Backus–Naur Form), a traditional notation for writing CFGs
- Now the grammar becomes simpler. Let:
 - $\Sigma = \{ \texttt{'[}', \texttt{']'}, \texttt{','}, \texttt{INT} \}$
 - $N = \{ \texttt{List}, \texttt{Elements} \}$
 - $S_0 = \texttt{List}$

```

List    → '[' Elements ']' | '[' ']'
Elements → INT | INT ',' Elements

```

- This grammar describes the same language at the syntactic level, but the parser no longer has to reconstruct integers from individual digits

5.2 Top-down Parsing

- **Top-down parsing** constructs a parse tree starting from the start symbol and working toward the input
- At each step, the parser expands a nonterminal according to one of its productions
- The goal is to choose productions so that the generated sentential form matches the input token stream

Predictive parsing

- A parser must decide which production to use when expanding a nonterminal
- For the parser to be efficient, this choice should be made **predictively**:
 - it should choose the correct production without backtracking
 - it should rely only on a bounded amount of lookahead
 - this yields a parsing procedure with linear running time
- To do this, the parser inspects one or more upcoming input tokens, called the **lookahead**

LL(k) parsing

- An **LL(k) parser** is a *predictive top-down* parser that:
 - reads input **Left-to-right**
 - constructs a **Leftmost** derivation
 - uses k lookahead tokens
- In particular, an **LL(1) parser** uses just one lookahead token to decide which production to apply
 - We will focus on this class **LL(1)**

LL(k) grammars and languages

- Not every grammar can be parsed by an LL(k) parser
 - A grammar that can be parsed this way is called an **LL(k) grammar**
 - A grammar may fail to be LL(n) for some n , but still be LL(m) for some $m > n$

- In some cases, a grammar that is not $LL(k)$ can be rewritten into an equivalent grammar that is $LL(k)$
 - Typical transformations include removing left recursion and left factoring
- However, not every context-free language is an $LL(k)$ language for some finite k
 - A language that can be generated by some $LL(k)$ grammar is called an **$LL(k)$ language**

Advantages and limitations

- LL parsers are attractive because they are:
 - simple to implement
 - efficient, usually linear-time parsing
 - well suited to many programming-language grammars

However, they also have limitations:

- many grammars are not $LL(k)$ for any finite k
- some languages are not $LL(k)$ for any finite k
- even when a language is $LL(k)$, its most natural grammar may not be
- grammar transformations are often required before predictive parsing is possible

✓ $LL(1)$ grammars

- An $LL(1)$ parser is a predictive top-down parser that:
 - reads input Left-to-right
 - produces a Leftmost derivation
 - uses **1 lookahead token**
- To decide which production to apply, the parser uses **lookahead sets**
- To compute these sets we define, for each nonterminal $A \in N$:
 - **FIRST(A)** - terminals that can appear at the beginning of strings derived from A
 - **FOLLOWS(A)** - terminals that can appear immediately after A

FIRST sets

$$\text{FIRST}(\mathbf{a}) = \{\mathbf{a}\}, \quad \text{if } \mathbf{a} \in \Sigma$$

$$\text{FIRST}(\varepsilon) = \{\varepsilon\}$$

$$\text{FIRST}(\mathbf{A}) = \bigcup_{\mathbf{A} \rightarrow \mathbf{x}_1 \mathbf{x}_2 \dots \mathbf{x}_n \in P} \text{FIRST}(\mathbf{x}_1 \mathbf{x}_2 \dots \mathbf{x}_n)$$

That is, the **FIRST** set of a terminal or ε is the symbol itself; for a nonterminal, one must calculate the **FIRST** set of all its productions.

$$\begin{aligned} \text{FIRST}(\mathbf{x}_1 \mathbf{x}_2 \dots \mathbf{x}_n) &= (\text{FIRST}(\mathbf{x}_1) \setminus \{\varepsilon\}) \\ &\cup \begin{cases} \text{FIRST}(\mathbf{x}_2 \dots \mathbf{x}_n) & \text{if } \varepsilon \in \text{FIRST}(\mathbf{x}_1) \\ \emptyset & \text{otherwise} \end{cases} \end{aligned}$$

That is, the **FIRST** set of a (non-empty) production is the **FIRST** set of the first symbol of the production and, if that first symbol is *nullable*, the **FIRST** set of the production after that symbol.

 **FIRST** sets example: A language for lists

- Revisiting the CFG:

```
p1: List → '[' Elements ']'
p2:      | '[' ']'
p3: Elements → INT
p4:      | INT ',' Elements
```

- For nonterminal **List**, there are two productions:

$$\begin{aligned} \text{FIRST}(\texttt{ '[' Elements ' '}) &= \text{FIRST}(\texttt{ '['}) \cup \text{FIRST}(\texttt{ '[' Elements ' '}) \\ &= \text{FIRST}(\texttt{ '['}) = \{\texttt{ '['}\} \cup \text{FIRST}(\texttt{ '[' Elements ' '}) \\ &= \text{FIRST}(\texttt{ '['}) = \{\texttt{ '['}\} \end{aligned}$$

$$\begin{aligned} \text{FIRST}(\texttt{ '[' Elements ' '}) &= \text{FIRST}(\texttt{ '[' Elements ' '}) \\ &= \text{FIRST}(\texttt{ '['}) \cup \text{FIRST}(\texttt{ '[' Elements ' '}) \\ &= \text{FIRST}(\texttt{ '['}) = \{\texttt{ '['}\} \end{aligned}$$

- For nonterminal **Elements** there are also two productions:

$$\begin{aligned} \text{FIRST}(\texttt{ INT}) &= \{\texttt{ INT}\} \\ \text{FIRST}(\texttt{ INT ',' Elements}) &= \text{FIRST}(\texttt{ INT}) \cup \text{FIRST}(\texttt{ INT ',' Elements}) \\ &= \{\texttt{ INT}\} \cup \text{FIRST}(\texttt{ INT ',' Elements}) \\ &= \{\texttt{ INT}\} \cup \text{FIRST}(\texttt{ INT}) \\ &= \{\texttt{ INT}\} \end{aligned}$$

FOLLOW sets

FOLLOW(A) sets are calculated as a *fixed point*.

- Let S be the grammar's start symbol, and $\$ \in \Sigma$ be a special terminal symbol representing the end of input. Then the process starts with

$$\text{FOLLOW}(S) = \{\$ \}$$

- Afterwards, for any nonterminal B , search what may follow it when it appears on the right-hand side of productions and expand its FOLLOW set.

$$\text{If } A \rightarrow X_1 \dots X_n B Y_1 \dots Y_m, \text{ then } \text{FOLLOW}(B) \supseteq$$

$$\text{If } A \rightarrow X_1 \dots X_n B Y_1 \dots Y_m \text{ and } \varepsilon \in \text{FIRST}(Y_1 \dots Y_m), \text{ then } \text{FOLLOW}(B) \supseteq$$

$$\text{If } A \rightarrow X_1 \dots X_n B, \text{ then } \text{FOLLOW}(B) \supseteq$$

 FOLLOW sets example: A language for lists

- The start symbol is $\texttt{List}$ and it does not appear in the right-hand side of any production
 - $\text{FOLLOW}(\texttt{List}) = \{ \$ \}$
- The nonterminal $\texttt{Elements}$ appears in the right-hand side of two productions
 - $\texttt{List} \rightarrow '[' \texttt{Elements} ']'$, so $\texttt{'}'$ belongs to $\text{FOLLOW}(\texttt{Elements})$
 - $\texttt{Elements} \rightarrow \texttt{INT} ', ' \texttt{Elements}$, but since it appears last, we must calculate $\text{FOLLOW}(\texttt{Elements})$; that is what we were already calculating, so we have reached the fixed point
 - $\text{FOLLOW}(\texttt{Elements}) = \{ \texttt{'}' \}$

LL(1) predictive parsing table

- The predictive parsing table will tell the parser which production to apply when expanding a nonterminal A given the lookahead token a

- A nonterminal A with lookahead a is expanded with production p for A if, and only if:
 - $a \in \text{FIRST}(p)$ or
 - $\epsilon \in \text{FIRST}(p)$ and $a \in \text{FOLLOW}(A)$

LL(1) conflicts

- For a nonterminal A and a lookahead a :
 - If there are two productions p_1 and p_2 for A such that $\text{FIRST}(p_1) \cap \text{FIRST}(p_2) \neq \emptyset$, then there is a **$\text{FIRST}/\text{FIRST}$ conflict**
 - Left-recursion in a production always leads to **$\text{FIRST}/\text{FIRST}$ conflict**
 - If a production p_1 for A is nullable ($\epsilon \in \text{FIRST}(p_1)$) and $\text{FOLLOW}(A) \cap \text{FIRST}(p_2) \neq \emptyset$ for another production p_2 , then there is a **$\text{FIRST}/\text{FOLLOW}$ conflict**
- The **LL(1) condition** is, thus, that for every nonterminal A , and productions p_i and p_j for A :

$$\begin{aligned} & \text{FIRST}(p_i) \cap \text{FIRST}(p_j) = \emptyset, \text{ if } i \neq j \\ & \text{FIRST}(p_i) \cap \text{FOLLOW}(A) = \emptyset, \text{ if } \epsilon \in \text{FIRST}(p_i) \end{aligned}$$
- A grammar is LL(1) if there are no **$\text{FIRST}/\text{FIRST}$** nor **$\text{FIRST}/\text{FOLLOW}$** conflicts

LL(1) conflicts example: A language for lists

- For nonterminal `List`, $\mathrm{FIRST}(p_1) \cap \mathrm{FIRST}(p_2) = \{\texttt{'['}\} \neq \emptyset$, there is a $\mathrm{FIRST}/\mathrm{FIRST}$ conflict
- For nonterminal `Elements`, $\mathrm{FIRST}(p_3) \cap \mathrm{FIRST}(p_4) = \{\texttt{INT}\} \neq \emptyset$, there is a $\mathrm{FIRST}/\mathrm{FIRST}$ conflict
- Thus, the grammar is not LL(1), and it is impossible to implement an LL(1) parser for it:
 - When expanding `List`, it is not possible to decide whether to apply `p1` or `p2`
 - Likewise for `Elements` and `p3` or `p4`

LL(1) rewriting

- Often, if the language is LL(1), we can convert a non-LL(1) grammar into a LL(1) version by:
 - Removing left-recursion
 - Factoring out common first elements
- This may result in a grammar parsable by top-down strategies, but less readable to humans
- But recall that not all languages are LL(1)

✓ LL(1) rewriting example: A language for lists

- The CFG has no left recursion, so let's try to factor out the first symbols to fix the $\mathrm{FIRST}(\mathrm{FIRST})/\mathrm{FIRST}(\mathrm{FIRST})$ conflicts

```
p1: List      → '[' RestList
p2: RestList  → Elements ']'
p3:           | ']'
p4: Elements  → INT RestElements
p5: RestElements → ',' Elements
p6: RestElements | ε
```

- Let us calculate again the $\mathrm{FIRST}(\mathrm{FIRST})$ and $\mathrm{FOLLOW}(\mathrm{FOLLOW})$ sets

- For nonterminal **List**:

$$\begin{aligned} \mathrm{FIRST}(\mathrm{List}) &= \mathrm{FIRST}([' \mathrm{RestList}]) = \{ '[' \} \\ \mathrm{FIRST}(\mathrm{List}) &= \{ '[' \} \end{aligned}$$

- For nonterminal **RestList**:

$$\begin{aligned} \mathrm{FIRST}(\mathrm{RestList}) &= \mathrm{FIRST}(\mathrm{Elements} \mathrm{'])} = \\ &= \mathrm{FIRST}(\mathrm{Elements}) = \mathrm{FIRST}(\mathrm{INT} \backslash \\ &\quad \mathrm{RestElements}) = \{ \mathrm{INT} \} \backslash \backslash \mathrm{FIRST}(\mathrm{'])} = \{ \\ &\quad \mathrm{'])} \} \end{aligned}$$

$$\begin{aligned} \mathrm{FIRST}(\mathrm{RestList}) &= \mathrm{FIRST}(\mathrm{Elements} \backslash \\ &\quad \mathrm{'])} = \mathrm{FIRST}(\mathrm{Elements}) = \mathrm{FIRST} \\ &\quad (\mathrm{INT} \backslash \mathrm{RestElements}) = \{ \mathrm{INT} \} \backslash \backslash \mathrm{FIRST} \\ &\quad (\mathrm{'])} = \{ \mathrm{'])} \} \end{aligned}$$

- No conflict for **RestList** since $\mathrm{FIRST}(\mathrm{RestList}) \cap \mathrm{FIRST}(\mathrm{RestList}) = \emptyset$

- For nonterminal **Elements**:

$$\begin{aligned} \mathrm{FIRST}(\mathrm{Elements}) &= \mathrm{FIRST}(\mathrm{INT} \backslash \mathrm{RestElements}) = \{ \\ &\quad \mathrm{INT} \} \end{aligned}$$

- For nonterminal **RestElements**:

$$\begin{aligned} \mathrm{FIRST}(\mathrm{RestElements}) &= \mathrm{FIRST}(\mathrm{') \text{Elements}}) = \{ \mathrm{') \} \\ &\quad \mathrm{FIRST}(\epsilon) = \{ \epsilon \} \end{aligned}$$

$\mathrm{FIRST}(\mathtt{' , '}) = \{ \mathtt{' , ' } \}$

$\mathrm{FIRST}(\epsilon) = \{ \epsilon \}$

- No

$\mathrm{FIRST}(\mathrm{FIRST})/\mathrm{FIRST}$

conflict, but since `RestElements` is nullable ($\epsilon \in \mathrm{FIRST}(p_6)$), we also need to consider $\mathrm{FOLLOW}(\mathtt{RestElements})$

$\mathrm{FOLLOW}(\mathtt{RestElements})$

- `RestElements` only appears in the right-hand side of `p4: Elements → INT RestElements`

- Since it is the last symbol, we must now find $\mathrm{FOLLOW}(\mathtt{Elements})$

- `Elements` appears in the right-hand side of `p2` and `p5`

- From `RestList → Elements ']'` we obtain `']'` in $\mathrm{FOLLOW}(\mathtt{Elements})$

- From `RestElements → ', ' Elements`, since `Elements` is last, we add $\mathrm{FOLLOW}(\mathtt{RestElements})$ to $\mathrm{FOLLOW}(\mathtt{Elements})$, which was the starting point

- So $\mathrm{FOLLOW}(\mathtt{RestElements}) = \{ \mathtt{' , ' } \}$

- Since $\mathrm{FIRST}(\mathtt{' , '}) \cap \mathrm{FOLLOW}(\mathtt{RestElements}) = \emptyset$ and $\mathrm{FIRST}(\epsilon) \cap \mathrm{FOLLOW}(\mathtt{RestElements}) = \emptyset$, there are no $\mathrm{FIRST}(\mathrm{FIRST})/\mathrm{FOLLOW}$ conflicts

LL(1) parsing table example: A language for lists

- Now that we have shown that the CFG is LL(1), we can use the $\mathrm{FIRST}(\mathrm{FIRST})$ and $\mathrm{FIRST}(\mathrm{FIRST})$ sets to construct the parsing table

- All entries use the FIRST set of productions except for `RestElements` with lookahead `]`: since `p6` is nullable, the FOLLOW set was used

	[	]	,	INT	\$
List	p1				
RestList		p3		p2	
Elements				p4	
RestElements		p6	p5		

✓ Implementing predictive parsing

The predictive choices encoded in an LL(1) parsing table can be implemented in two main ways:

- recursive descent parsing
- table-driven parsing

Recursive descent parsing

- A recursive descent parser implements predictive parsing using functions:
 - each nonterminal corresponds to one function
 - each production corresponds to one branch inside that function
- Recursive descent parsers:
 - closely mirror the structure of the grammar
 - are simple to implement by hand

Processing terminals

- To recognize a terminal, we check whether the next token has the expected type
- We could have a function `recognize_a()` for each token type `a`, but a generic helper function is enough:

```
def recognize_terminal(expected_type):
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == expected_type:
        next_token()
        return token_val
```

```

else:
    raise ParseError(f"Unexpected token when recognizing terminal {expected}")

```

✓ Processing nonterminals

- For a nonterminal A , we create one branch for each relevant parsing table entry
- Suppose

```

p1: A → X1 X2 ... Xn
px: A → ...
pk: A → Y1 Y2 ... Ym

```

and the parsing table says A expands with $p1$ with lookahead x , and with pn with lookahead y

- The skeleton for the function for recognizing A would be:

```

def recognize_A():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "x":
        # p1: A → X1 X2 ... Xn
        x1 = recognize_X1()
        x2 = recognize_X2()
        ...
        xn = recognize_Xn()
        return process(x1,x2,...,xn)
    elif ...:
        ...
    elif token_type == "y":
        # pk: A → Y1 Y2 ... Ym
        y1 = recognize_Y1()
        y2 = recognize_Y2()
        ...
        ym = recognize_Ym()
        return process(y1,y2,...,ym)
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal A: {token_val}")

```

- The values returned by these functions depend on the goal of the parser:

- If we only want to **recognize** the input (check whether it is syntactically valid), the functions do not need to return anything
- If we want to **build an abstract syntax tree (AST)**, each branch should return the corresponding node
- Alternatively, the parser can perform **semantic actions**, such as interpreting or evaluating the code while parsing

✓ Recursive descent parser example: A language for lists

Let us define recursive descent parser for the following LL(1) grammar:

```
p1: List      → '[' RestList
p2: RestList  → Elements ']'
p3:           → ']'
p4: Elements  → INT RestElements
p5: RestElements → ',' Elements
p6:           → ε
```

- First, we define the lexer (using `Ply.lex`)

```
!pip install ply
```

[Mostrar saída oculta](#)

```
import ply.lex as lex

class LexError(Exception):
    pass

tokens = ("INT",)
literals = "[],"

def t_INT(t):
    r"\d+"
    t.value = int(t.value)
    return t

t_ignore = " \t\n"

def t_error(t):
    raise LexError(f"Invalid symbol: {t.value[0]}")

__file__ = "Untitled.ipynb" # needed to run PLY inside notebook
lexer = lex.lex()

_lookahead = None
```

```
# asks the lexer for the next token
def next_token():
    global _lookahead
    _lookahead = lexer.token()

# auxiliary function for token type, returns special end of input $ token if
def lookahead():
    return (_lookahead.type, _lookahead.value, _lookahead.lineno, _lookahead.lexp
```

- Next, define the function to recognize terminals

```
class ParseError(Exception):
    pass

# generic function to recognize terminals
def recognize_terminal(expected_type):
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == expected_type:
        next_token()
        return token_val
    else:
        raise ParseError(f"Unexpected token when recognizing terminal {expected_
```

- Lastly, using the parsing table defined above, define the functions to recognize nonterminals

```
def recognize_List():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "[": # p1: List → '[' RestList
        print("Applying p1: List → '[' RestList")
        recognize_terminal("[")
        recognize_RestList()
        print("Recognized p1: List → '[' RestList")
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'List':

def recognize_RestList():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "INT": # p2: RestList → Elements ']'
        print("Applying p2: RestList → Elements ']'")
        recognize_Elements()
        recognize_terminal("]")
        print("Recognized p2: RestList → Elements ']'")
    elif token_type == "]": # p3: | ']'
        print("Applying p3: | ']'")
        recognize_terminal("]")
        print("Recognized p3: | ']'")
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'RestLi

def recognize_Elements():
```

```

token_type, token_val, token_line, token_pos = lookahead()
if token_type == "INT":    # p4: Elements      → INT RestElements
    print("Applying p4: Elements      → INT RestElements")
    recognize_terminal("INT")
    recognize_RestElements()
    print("Recognized p4: Elements      → INT RestElements")
else:
    raise ParseError(f"Unexpected token when recognizing nonterminal 'Elemen

def recognize_RestElements():
    token_type, token_val, token_line, token_col = lookahead()
    if token_type == ",":    # p5: RestElements → ',' Elements
        print("Applying p5: RestElements → ',' Elements")
        recognize_terminal(",")
        recognize_Elements()
        print("Recognized p5: RestElements → ',' Elements")
    elif token_type == "]":    # p6:                | ε
        print("Applying p6:                | ε")
        pass
        print("Recognized p6:                | ε")
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'RestEl

```

- Now, start from the initial symbol, `List`, and check whether the input is exactly recognized as a `List`

```

def parse(text):
    lexer.input(text)
    next_token()
    recognize_List()
    if lookahead()[0] != "$":
        raise ParseError(f"Unexpected extra input: {lookahead()[0]}")

```

```

parse("[10, 20, 30]")

```

```

Applying p1: List      → '[' RestList
Applying p2: RestList  → Elements ']'
Applying p4: Elements  → INT RestElements
Applying p5: RestElements → ',' Elements
Applying p4: Elements  → INT RestElements
Applying p5: RestElements → ',' Elements
Applying p4: Elements  → INT RestElements
Applying p6:          | ε
Recognized p6:        | ε
Recognized p4: Elements → INT RestElements
Recognized p5: RestElements → ',' Elements
Recognized p4: Elements → INT RestElements
Recognized p5: RestElements → ',' Elements
Recognized p4: Elements → INT RestElements
Recognized p2: RestList  → Elements ']'
Recognized p1: List      → '[' RestList

```

parse("[10]")

Applying p1: List	→ '[' RestList
Applying p2: RestList	→ Elements '']'
Applying p4: Elements	→ INT RestElements
Applying p6:	ε
Recognized p6:	ε
Recognized p4: Elements	→ INT RestElements
Recognized p2: RestList	→ Elements '']'
Recognized p1: List	→ '[' RestList

parse("[]")

Applying p1: List	→ '[' RestList
Applying p3:	'']'
Recognized p3:	'']'
Recognized p1: List	→ '[' RestList

parse("[10,]")


```

Applying p1: List      → '[' RestList
Applying p2: RestList  → Elements ']'
Applying p4: Elements  → INT RestElements
Applying p5: RestElements → ',' Elements

```

ParseError Traceback (most recent call last)

Untitled.ipynb in <cell line: 0>()

```
----> 1 parse("[10, ]")
```

Untitled.ipynb in parse(text)

```

    2 lexer.input(text)
    3 next_token()
----> 4 recognize_List()
    5 if lookahead()[0] != "$":
    6     raise ParseError(f"Unexpected extra input: {lookahead()[0]}")

```

Untitled.ipynb in recognize_List()

```

    4 print("Applying p1: List      → '[' RestList")
    5 recognize_terminal("[")
----> 6 recognize_RestList()
    7 print("Recognized p1: List    → '[' RestList")
    8 else:

```

Untitled.ipynb in recognize_RestList()

```

    13 if token_type == "INT": # p2: RestList → Elements ']'
    14     print("Applying p2: RestList → Elements ']'")
---> 15 recognize_Elements()
    16 recognize_terminal("]")
    17 print("Recognized p2: RestList → Elements ']'")

```

Untitled.ipynb in recognize_Elements()

```

    28 print("Applying p4: Elements → INT RestElements")
    29 recognize_terminal("INT")
---> 30 recognize_RestElements()
    31 print("Recognized p4: Elements → INT RestElements")
    32 else:

```

Untitled.ipynb in recognize_RestElements()

```

    38 print("Applying p5: RestElements → ',' Elements")
    39 recognize_terminal(",")
---> 40 recognize_Elements()
    41 print("Recognized p5: RestElements → ',' Elements")
    42 elif token_type == "]: # p6:          | ε

```

Untitled.ipynb in recognize_Elements()

```

    31 print("Recognized p4: Elements → INT RestElements")
    32 else:
---> 33 raise ParseError(f"Unexpected token when recognizing nonterminal
'Elements': {token_type}")
    34
    35 def recognize_RestElements():

```

ParseError: Unexpected token when recognizing nonterminal 'Elements':]

parse("[10, 20")

```
Applying p1: List      → '[' RestList
Applying p2: RestList  → Elements ']'
Applying p4: Elements  → INT RestElements
Applying p5: RestElements → ',' Elements
Applying p4: Elements  → INT RestElements
```

ParseError Traceback (most recent call last)

```
Untitled.ipynb in <cell line: 0>()
```

```
----> 1 parse("[10, 20")
```

```
Untitled.ipynb in parse(text)
```

```
    2 lexer.input(text)
    3 next_token()
----> 4 recognize_List()
    5 if lookahead()[0] != "$":
    6     raise ParseError(f"Unexpected extra input: {lookahead()[0]}")
```

```
Untitled.ipynb in recognize_List()
```

```
    4 print("Applying p1: List      → '[' RestList")
    5 recognize_terminal("[")
----> 6 recognize_RestList()
    7 print("Recognized p1: List    → '[' RestList")
    8 else:
```

```
Untitled.ipynb in recognize_RestList()
```

```
   13 if token_type == "INT": # p2: RestList → Elements ']'
   14     print("Applying p2: RestList → Elements ']'")
---> 15 recognize_Elements()
   16 recognize_terminal("]")
   17 print("Recognized p2: RestList → Elements ']'")
```

```
Untitled.ipynb in recognize_Elements()
```

```
   28 print("Applying p4: Elements → INT RestElements")
   29 recognize_terminal("INT")
---> 30 recognize_RestElements()
   31 print("Recognized p4: Elements → INT RestElements")
   32 else:
```

```
Untitled.ipynb in recognize_RestElements()
```

```
   38 print("Applying p5: RestElements → ',' Elements")
   39 recognize_terminal(",")
---> 40 recognize_Elements()
   41 print("Recognized p5: RestElements → ',' Elements")
   42 elif token_type == "]: # p6:          | ε
```

```
Untitled.ipynb in recognize_Elements()
```

```
   28 print("Applying p4: Elements → INT RestElements")
   29 recognize_terminal("INT")
---> 30 recognize_RestElements()
   31 print("Recognized p4: Elements → INT RestElements")
   32 else:
```

```
Untitled.ipynb in recognize_RestElements()
```

```
   45 print("Recognized p6:          | ε")
   46 else:
---> 47 raise ParseError(f"Unexpected token when recognizing
nonterminal 'RestElements': {token_type}")
```

```
parse("[10 20]")
```

```
Applying p1: List      → '[' RestList
Applying p2: RestList  → Elements ']'
Applying p4: Elements  → INT RestElements
```

ParseError Traceback (most recent call last)

```
Untitled.ipynb in <cell line: 0>()
```

```
----> 1 parse("[10 20]")
```

```
Untitled.ipynb in parse(text)
```

```
2 lexer.input(text)
```

```
3 next_token()
```

```
----> 4 recognize_List()
```

```
5 if lookahead()[0] != "$":
```

```
6     raise ParseError(f"Unexpected extra input: {lookahead()[0]}")
```

```
Untitled.ipynb in recognize_List()
```

```
4 print("Applying p1: List      → '[' RestList")
```

```
5 recognize_terminal("[")
```

```
----> 6 recognize_RestList()
```

```
7 print("Recognized p1: List      → '[' RestList")
```

```
8 else:
```

```
Untitled.ipynb in recognize_RestList()
```

```
13 if token_type == "INT": # p2: RestList → Elements ']'
```

```
14 print("Applying p2: RestList → Elements ']'")
```

```
---> 15 recognize_Elements()
```

```
16 recognize_terminal("]")
```

```
17 print("Recognized p2: RestList → Elements ']'")
```

```
Untitled.ipynb in recognize_Elements()
```

```
28 print("Applying p4: Elements → INT RestElements")
```

```
29 recognize_terminal("INT")
```

```
---> 30 recognize_RestElements()
```

```
31 print("Recognized p4: Elements → INT RestElements")
```

```
32 else:
```

```
Untitled.ipynb in recognize_RestElements()
```

```
45 print("Recognized p6:      | ε")
```

```
46 else:
```

```
---> 47 raise ParseError(f"Unexpected token when recognizing nonterminal  
'RestElements': {token_type}")
```

ParseError: Unexpected token when recognizing nonterminal 'RestElements':
INT

✓ Table-driven predictive parser

- Instead of hardcoding the decisions in code, the parser can consult the table directly
- The parser maintains:
 - a stack of grammar symbols
 - a lookahead token

- At each step:
 - inspect the stack top
 - inspect the lookahead token
 - consult the table
 - apply the corresponding production

```
def generate(start,terminals,parse_table,productions):
    def parser(text):
        lexer.input(text)
        next_token()
        stack = ["$", start]

        while stack:
            next_symbol = stack.pop()
            token_type, token_val, token_line, token_pos = lookahead()

            if next_symbol in terminals:
                if next_symbol == token_type:
                    next_token()
                else:
                    raise ParseError(f"Unexpected token when recognizing terminal {n
            else:
                next_production = parse_table.get((next_symbol,token_type))
                if not next_production:
                    raise ParseError(f"Unexpected token when recognizing nonterminal {
                else:
                    rhs = productions[next_production]
                    print(f"Applying {next_production}: {next_symbol} → {rhs}")
                    stack.extend(rhs[::-1])

        return parser
```

✓ Table-driven predictive parser example: A language for lists

Start by encoding the parsing table and the production rules right-hand side as a dictionaries.

```
p1: List      → '[' RestList
p2: RestList  → Elements ']'
p3:           | ']'
p4: Elements  → INT RestElements
p5: RestElements → ',' Elements
p6:           | ε
```

	[	]	,	INT	\$
List	p1				
RestList		p3	p2		
Elements			p4		

```

parse_table = {
    ("List","[" ) : "p1",
    ("RestList","]") : "p3",
    ("RestList","INT") : "p2",
    ("Elements","INT") : "p4",
    ("RestElements",",") : "p5",
    ("RestElements","]") : "p6"
}

terminals = ("[" , "]" , "," , "INT" , "$")

productions = {
    "p1": ["[" , "RestList"],
    "p2": ["Elements" , "INT"],
    "p3": ["]"],
    "p4": ["INT" , "RestElements"],
    "p5": ["," , "Elements"],
    "p6": []
}

```

```

parse = generate("List",terminals,parse_table,productions)

```

```

parse("[10, 20, 30]")

```

```

Applying p1: List → ['[', 'RestList']
Applying p2: RestList → ['Elements', 'INT']
Applying p4: Elements → ['INT', 'RestElements']
Applying p5: RestElements → [',', 'Elements']
Applying p4: Elements → ['INT', 'RestElements']
Applying p5: RestElements → [',', 'Elements']
Applying p4: Elements → ['INT', 'RestElements']
Applying p6: RestElements → []

```

```

parse("[10]")

```

```

Applying p1: List → ['[', 'RestList']
Applying p2: RestList → ['Elements', 'INT']
Applying p4: Elements → ['INT', 'RestElements']
Applying p6: RestElements → []

```

```

parse("[]")

```

```

Applying p1: List → ['[', 'RestList']
Applying p3: RestList → [']']

```

```

parse("[10, ]")

```

```
Applying p1: List → [' ', 'RestList']
Applying p2: RestList → ['Elements', '']
Applying p4: Elements → ['INT', 'RestElements']
Applying p5: RestElements → ['', 'Elements']
```

```
-----
ParseError                                Traceback (most recent call last)
Untitled.ipynb in <cell line: 0>()
----> 1 parse("[10, ]")
```

```
Untitled.ipynb in parser(text)
    17         next_production = parse_table.get((next_symbol, token_type))
    18         if not next_production:
----> 19             raise ParseError(f"Unexpected token when recognizing
nonterminal {next_symbol}: {token_type}")
    20         else:
    21             rhs = productions[next_production]
```

```
ParseError: Unexpected token when recognizing nonterminal Elements: ]
```

```
parse("[10, 20"])
```

```
Applying p1: List → [' ', 'RestList']
Applying p2: RestList → ['Elements', '']
Applying p4: Elements → ['INT', 'RestElements']
Applying p5: RestElements → ['', 'Elements']
Applying p4: Elements → ['INT', 'RestElements']
```

```
-----
ParseError                                Traceback (most recent call last)
Untitled.ipynb in <cell line: 0>()
----> 1 parse("[10, 20"])
```

```
Untitled.ipynb in parser(text)
    17         next_production = parse_table.get((next_symbol, token_type))
    18         if not next_production:
----> 19             raise ParseError(f"Unexpected token when recognizing
nonterminal {next_symbol}: {token_type}")
    20         else:
    21             rhs = productions[next_production]
```

```
ParseError: Unexpected token when recognizing nonterminal RestElements: $
```

```
parse("[10 20]")
```

```

Applying p1: List → [' ', 'RestList']
Applying p2: RestList → ['Elements', '']
Applying p4: Elements → ['INT', 'RestElements']
-----
ParseError                                Traceback (most recent call last)
Untitled.ipynb in <cell line: 0>()
----> 1 parse("[10 20]")

Untitled.ipynb in parser(text)
      17         next_production = parse_table.get((next_symbol, token_type))
      18         if not next_production:
----> 19             raise ParseError(f"Unexpected token when recognizing
nonterminal {next_symbol}: {token_type}")
      20         else:
      21             rhs = productions[next_production]

ParseError: Unexpected token when recognizing nonterminal RestElements: INT

```

✓ ✂ LL(1) exercise: Simple arithmetic expressions

- Consider the following CFG:

```

p1: E → E + T
p2:   | T
p3: T → T * F
p4:   | F
p5: F → INT

```

- This grammar expresses:
 - $*$ has higher precedence than $+$
 - both operators are left-associative
- This grammar is good for expressing structure, but is it suitable for LL(1) parsing?
 - Check the LL(1) condition for the CFG
 - If the condition fails, rewrite it to a LL(1) version
 - Implement a LL(1) parser for the language generated by the grammar
- The CFG has left recursion in productions $p1$ and $p3$
- Immediate left recursion prevents LL(1) parsing, because productions for the same nonterminal have overlapping predictive choices
- We can remove left recursion by introducing new productions

```

p1: E → T ERest
p2: ERest → '+' T ERest

```

```

p3:      | ε
p4: T     → F TRest
p5: TRest → '*' F TRest
p6:      | ε
p7: F     → INT

```

- This grammar is equivalent (generates the same strings) but the resulting parse trees are different
 - In particular, it no longer associates to the left
 - This is not problematic because $+$ and $*$ are associative
 - But would be if we also supported $-$ and $/$

- FIRST sets

```

\begin{align}
&\text{FIRST}(\text{E}) = \{ \text{INT} \} \\
&\text{FIRST}(\text{E}') = \{ + \} \\
&\text{FIRST}(\epsilon) = \{ \epsilon \} \\
&\text{FIRST}(\text{T}) = \{ \text{INT} \} \\
&\text{FIRST}(\text{T}') = \{ * \} \\
&\text{FIRST}(\epsilon) = \{ \epsilon \} \\
&\text{FIRST}(\text{INT}) = \{ \text{INT} \} \\
\end{align}
\begin{align}
&\text{FIRST}(\text{T}) = \{ \text{INT} \} \\
&\text{FIRST}(\text{E}') = \{ + \} \\
&\text{FIRST}(\epsilon) = \{ \epsilon \} \\
&\text{FIRST}(\text{T}) = \{ \text{INT} \} \\
&\text{FIRST}(\text{T}') = \{ * \} \\
&\text{FIRST}(\epsilon) = \{ \epsilon \} \\
&\text{FIRST}(\text{INT}) = \{ \text{INT} \} \\
\end{align}

```

- FOLLOW sets

```

\begin{align}
&\text{FOLLOW}(\text{E}) = \{ \$ \} \\
&\text{FOLLOW}(\text{E}') = \{ \$ \} \\
&\text{FOLLOW}(\text{T}') = \{ +, \$ \} \\
&\text{FOLLOW}(\text{F}) = \{ *, +, \$ \} \\
\end{align}
\begin{align}
&\text{FOLLOW}(\text{E}) = \{ \$ \} \\
&\text{FOLLOW}(\text{E}') = \{ \$ \} \\
&\text{FOLLOW}(\text{T}) = \{ +, \$ \} \\
&\text{FOLLOW}(\text{T}') = \{ +, \$ \} \\
&\text{FOLLOW}(\text{F}) = \{ *, +, \$ \} \\
\end{align}

```

- Predictive parsing table

	$+$	$*$	INT	\$
E			p1	
E'	p2			p3
T			p4	
T'	p6	p5		p6
F			p7	


```

import ply.lex as lex

# -----
# Lexer
# -----

class LexError(Exception):
    pass

tokens = ("INT",)
literals = "+*"

def t_INT(t):
    r"\d+"
    t.value = int(t.value)
    return t

t_ignore = " \t\n"

def t_error(t):
    raise LexError(f"Invalid symbol: {t.value[0]}")

__file__ = "Untitled.ipynb"    # sometimes needed by PLY in notebooks
lexer = lex.lex()

_lookahead = None

def next_token():
    global _lookahead
    _lookahead = lexer.token()

def lookahead():
    return (_lookahead.type, _lookahead.value, _lookahead.lineno, _lookahead.lexp

```

```

# -----
# Parser
# -----

class ParseError(Exception):
    pass

def recognize_terminal(expected_type):
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == expected_type:
        next_token()
        return token_val
    else:
        raise ParseError(f"Unexpected token when recognizing terminal {expected_

def recognize_E():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "INT":          # p1: E → T ERest
        print("Applying p1: E → T ERest")

```

```

        recognize_T()
        recognize_ERest()
        print("Recognized p1: E → T ERest")
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal E: {tok

def recognize_ERest():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "+":
        # p2: ERest → + T ERest
        print("Applying p2: ERest → + T ERest")
        recognize_terminal("+")
        recognize_T()
        recognize_ERest()
        print("Recognized p2: ERest → + T ERest")
    elif token_type == "$":
        # p3:      → ε
        print("Applying p3:      → ε")
        pass
        print("Recognized p3:      → ε")
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal E': {tok

def recognize_T():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "INT":
        # p4: T → F TRest
        print("Applying p4: T → F TRest")
        recognize_F()
        recognize_TRest()
        print("Recognized p4: T → F TRest")
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal T: {tok

def recognize_TRest():
    token_type, token_val, token_line, token_col = lookahead()
    if token_type == "*":
        # p5: TRest → * F TRest
        print("Applying p5: TRest → * F TRest")
        recognize_terminal("*")
        recognize_F()
        recognize_TRest()
        print("Recognized p5: TRest → * F TRest")
    elif token_type in ("+", "$"):
        # p6:      → ε
        print("Applying p6:      → ε")
        pass
        print("Recognized p6:      → ε")
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal T': {tok

def recognize_F():
    token_type, token_val, token_line, token_col = lookahead()
    if token_type == "INT":
        # p7: F → INT
        print("Applying p7: F → INT")
        recognize_terminal("INT")
        print("Applying p7: F → INT")
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal F: {tok

```

```
def parse(text):
    lexer.input(text)
    next_token()
    recognize_E()
    if lookahead()[0] != "$":
        raise ParseError(f"Unexpected extra input: {lookahead()[0]}")
```

```
parse("10 + 20")
```

```
Applying p1: E → T ERest
Applying p4: T → F TRest
Applying p7: F → INT
Applying p7: F → INT
Applying p6: → ε
Recognized p6: → ε
Recognized p4: T → F TRest
Applying p2: ERest → + T ERest
Applying p4: T → F TRest
Applying p7: F → INT
Applying p7: F → INT
Applying p6: → ε
Recognized p6: → ε
Recognized p4: T → F TRest
Applying p3: → ε
Recognized p3: → ε
Recognized p2: ERest → + T ERest
Recognized p1: E → T ERest
```

```
parse("10 * 23")
```

```
Applying p1: E → T ERest
Applying p4: T → F TRest
Applying p7: F → INT
Applying p7: F → INT
Applying p5: TRest → * F TRest
Applying p7: F → INT
Applying p7: F → INT
Applying p6: → ε
Recognized p6: → ε
Recognized p5: TRest → * F TRest
Recognized p4: T → F TRest
Applying p3: → ε
Recognized p3: → ε
Recognized p1: E → T ERest
```

```
parse("23 * 10 + 13")
```

```
Applying p1: E → T ERest
Applying p4: T → F TRest
Applying p7: F → INT
Applying p7: F → INT
Applying p5: TRest → * F TRest
Applying p7: F → INT
Applying p7: F → INT
Applying p6: → ε
Recognized p6: → ε
Recognized p5: TRest → * F TRest
```

Recognized p4: $T \rightarrow F \text{ TRest}$
Applying p2: $\text{ERest} \rightarrow + T \text{ ERest}$
Applying p4: $T \rightarrow F \text{ TRest}$
Applying p7: $F \rightarrow \text{INT}$
Applying p7: $F \rightarrow \text{INT}$
Applying p6: $\rightarrow \epsilon$
Recognized p6: $\rightarrow \epsilon$
Recognized p4: $T \rightarrow F \text{ TRest}$
Applying p3: $\rightarrow \epsilon$
Recognized p3: $\rightarrow \epsilon$
Recognized p2: $\text{ERest} \rightarrow + T \text{ ERest}$
Recognized p1: $E \rightarrow T \text{ ERest}$

`parse("23 ** 34")`

```
Applying p1: E → T ERest
Applying p4: T → F TRest
Applying p7: F → INT
Applying p7: F → INT
Applying p5: TRest → * F TRest
```

ParseError Traceback (most recent call last)

```
Untitled.ipynb in <cell line: 0>()
```

```
----> 1 parse("23 ** 34")
```

```
Untitled.ipynb in parse(text)
```

```
    76 lexer.input(text)
    77 next_token()
---> 78 recognize_E()
    79 if lookahead()[0] != "$":
    80     raise ParseError(f"Unexpected extra input: {lookahead()[0]}")
```

```
Untitled.ipynb in recognize_E()
```

```
    18 if token_type == "INT":          # p1: E → T ERest
    19     print("Applying p1: E → T ERest")
---> 20     recognize_T()
    21     recognize_ERest()
    22     print("Recognized p1: E → T ERest")
```

```
Untitled.ipynb in recognize_T()
```

```
    44     print("Applying p4: T → F TRest")
    45     recognize_F()
---> 46     recognize_TRest()
    47     print("Recognized p4: T → F TRest")
    48     else:
```

```
Untitled.ipynb in recognize_TRest()
```

```
    54     print("Applying p5: TRest → * F TRest")
    55     recognize_terminal("*")
---> 56     recognize_F()
    57     recognize_TRest()
    58     print("Recognized p5: TRest → * F TRest")
```

```
Untitled.ipynb in recognize_F()
```

```
    71     print("Applying p7: F → INT")
    72     else:
---> 73     raise ParseError(f"Unexpected token when recognizing nonterminal
F: {token_type}")
    74
    75 def parse(text):
```

ParseError: Unexpected token when recognizing nonterminal F: *

```
parse("12 * 23 +")
```

```

Applying p1: E → T ERest
Applying p4: T → F TRest
Applying p7: F → INT
Applying p7: F → INT
Applying p5: TRest → * F TRest
Applying p7: F → INT
Applying p7: F → INT
Applying p6: → ε
Recognized p6: → ε
Recognized p5: TRest → * F TRest
Recognized p4: T → F TRest
Applying p2: ERest → + T ERest

```

```

-----
ParseError                                Traceback (most recent call last)
Untitled.ipynb in <cell line: 0>()
----> 1 parse("12 * 23 +")

Untitled.ipynb in parse(text)
    76 lexer.input(text)
    77 next_token()
----> 78 recognize_E()
    79 if lookahead()[0] != "$":
    80     raise ParseError(f"Unexpected extra input: {lookahead()[0]}")

Untitled.ipynb in recognize_E()
    19 print("Applying p1: E → T ERest")
    20 recognize_T()
----> 21 recognize_ERest()
    22 print("Recognized p1: E → T ERest")
    23 else:

Untitled.ipynb in recognize_ERest()
    29 print("Applying p2: ERest → + T ERest")
    30 recognize_terminal("+")
----> 31 recognize_T()
    32 recognize_ERest()
    33 print("Recognized p2: ERest → + T ERest")

Untitled.ipynb in recognize_T()
    47 print("Recognized p4: T → F TRest")
    48 else:
----> 49     raise ParseError(f"Unexpected token when recognizing nonterminal
T: {token_type}")
    50
    51 def recognize_TRest():

ParseError: Unexpected token when recognizing nonterminal T: $

```

✓ 5.3 From recognition to structure

- So far, our parsers only recognize whether the input is **syntactically valid**
- For example:

```
[10, 20, 30]
```

- There is are syntactic errors, so our parser simply reports *parse succeeded* 
- However, most real parsers do more than recognition
- Instead of only checking validity, the parser usually computes or constructs information about the input

Returning values from parsing functions

- Previously, our functions only consumed tokens and applied productions; they returned nothing
- We can modify the parser so that functions return *semantic values associated with grammar symbols*
 - We call these **semantic actions**
- These values may represent:
 - computed results
 - structured data
 - intermediate representations

✓ Evaluation example: A language for lists

- Besides parsing our language for lists of integers, let us try to compute the *sum* of the list during parsing
- Recall the LL(1) grammar:

```
p1: List      → '[' RestList
p2: RestList  → Elements ']'
p3:           | ']'
p4: Elements  → INT RestElements
p5: RestElements → ',' Elements
p6:           | ε
```

- The lexer and auxiliary functions remain the same; we only adapt the functions recognizing nonterminals
 - Recall that tokens already have a *value* assigned during lexing

```
def recognize_List():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "[":      # p1: List      → '[' RestList
        recognize_terminal("[")
```

```

        total = recognize_RestList()
        return total          # return the total of the actual list
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'List':

def recognize_RestList():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "INT":    # p2: RestList      → Elements ']'
        total = recognize_Elements()
        recognize_terminal("]")
        return total          # has elements, return their total
    elif token_type == "]:":  # p3:                | ']'
        recognize_terminal("]")
        return 0              # no elements, total is 0
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'RestLi

def recognize_Elements():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "INT":    # p4: Elements      → INT RestElements
        elem = recognize_terminal("INT")
        elems = recognize_RestElements()
        return elem + elems    # an element plus the total of rest of the list
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'Elemen

def recognize_RestElements():
    token_type, token_val, token_line, token_col = lookahead()
    if token_type == ",":      # p5: RestElements → ',' Elements
        recognize_terminal(",")
        elems = recognize_Elements()
        return elems          # more elements, return their total
    elif token_type == "]:":  # p6:                | ε
        return 0              # end of the list, total is 0
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'RestEl

```

```

def parse(text):
    lexer.input(text)
    next_token()
    result = recognize_List() # store the parsing result
    if lookahead()[0] != "$":
        raise ParseError(f"Unexpected extra input: {lookahead()[0]}")
    return result

```

```
parse("[10, 20, 30]")
```

```
60
```

```
parse("[10]")
```

```
10
```



```
parse("[]")
```

```
0
```

✓ ✂ Evaluation exercise: Simple arithmetic expressions

- Recall the following LL(1) CFG from the previous exercise:

```
p1: E      → T ERest
p2: ERest → '+' T ERest
p3:       | ε
p4: T      → F TRest
p5: TRest  → '*' F TRest
p6:       | ε
p7: F      → INT
```

- Write a top-down predictive parser to calculate the value of such expressions

```
import ply.lex as lex

# -----
# Lexer
# -----

class LexError(Exception):
    pass

tokens = ("INT",)
literals = "+*"

def t_INT(t):
    r"\d+"
    t.value = int(t.value)
    return t

t_ignore = " \t\n"

def t_error(t):
    raise LexError(f"Invalid symbol: {t.value[0]}")

__file__ = "Untitled.ipynb"  # sometimes needed by PLY in notebooks
lexer = lex.lex()

_lookahead = None

def next_token():
    global _lookahead
    _lookahead = lexer.token()
```

```
def lookahead():
    return (_lookahead.type, _lookahead.value, _lookahead.lineno, _lookahead.lexp
```

```
# -----
# Parser
# -----
```

```
class ParseError(Exception):
    pass
```

```
def recognize_terminal(expected_type):
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == expected_type:
        next_token()
        return token_val
    else:
        raise ParseError(f"Unexpected token when recognizing terminal {expected_
```

```
def recognize_E():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "INT":          # p1: E → T ERest
        left = recognize_T()
        rest = recognize_ERest()
        return left + rest
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal E: {tok
```

```
def recognize_ERest():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "+":          # p2: ERest → + T ERest
        recognize_terminal("+")
        right = recognize_T()
        rest = recognize_ERest()
        return right + rest
    elif token_type == "$":        # p3:      → ε
        return 0
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal E': {to
```

```
def recognize_T():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "INT":        # p4: T → F TRest
        left = recognize_F()
        rest = recognize_TRest()
        return left * rest
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal T: {tok
```

```
def recognize_TRest():
    token_type, token_val, token_line, token_col = lookahead()
    if token_type == "*":          # p5: TRest → * F TRest
        recognize_terminal("*")
        right = recognize_F()
        rest = recognize_TRest()
```

```

        return right * rest
    elif token_type in ("+", "$"): # p6:  $\rightarrow \epsilon$ 
        return 1
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal T': {token}")

def recognize_F():
    token_type, token_val, token_line, token_col = lookahead()
    if token_type == "INT": # p7:  $F \rightarrow INT$ 
        val = recognize_terminal("INT")
        return val
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal F: {token}")

def parse(text):
    lexer.input(text)
    next_token()
    tree = recognize_E()
    if lookahead()[0] != "$":
        raise ParseError(f"Unexpected extra input: {lookahead()[0]}")
    return tree

```

```
parse("10 + 20")
```

```
30
```

```
parse("10 + 20 * 30")
```

```
610
```

```
parse("10 * 20 + 30")
```

```
230
```

```
parse("10")
```

```
10
```

Creating structured representations

- Often, we do not want to evaluate an input immediately
- Before evaluation, or generating the final representation, we may want to:
 - check for additional errors
 - perform additional translations
 - apply optimizations
- Therefore, we usually generate an **intermediate representation** that can be analyzed and transformed

✓ Structured representation example: A language for lists

- Instead of computing the sum, let us try to construct a **structured representation** of the list
- In this example, we can build an actual Python list while parsing
- Again, the lexer and auxiliary functions remain unchanged

```
def recognize_List():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "[":          # p1: List          → '[' RestList
        recognize_terminal("[")
        elems = recognize_RestList()
        return elems              # return the elements of the actual list
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'List':")

def recognize_RestList():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "INT":        # p2: RestList      → Elements ']'
        elems = recognize_Elements()
        recognize_terminal("]")    # has elements, return them
        return elems
    elif token_type == "]:        # p3:                | ']'
        recognize_terminal("]")    # has no elements, return empty list
        return []
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'RestLi")

def recognize_Elements():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "INT":        # p4: Elements      → INT RestElements
        elem = recognize_terminal("INT")
        elems = recognize_RestElements()
        return [elem] + elems      # element followed by the elements of the res
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'Elemen")

def recognize_RestElements():
    token_type, token_val, token_line, token_col = lookahead()
    if token_type == ",":          # p5: RestElements → ',' Elements
        recognize_terminal(",")
        elems = recognize_Elements()
        return elems              # has more elements, return them
    elif token_type == "]:        # p6:                | ε
        return []                 # no more elements, return empty list
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'RestEl")
```

```
def parse(text):
    lexer.input(text)
```

```
next_token()
result = recognize_List()    # store the parsing result
if lookahead()[0] != "$":
    raise ParseError(f"Unexpected extra input: {lookahead()[0]}")
return result
```

```
parse("[10, 20, 30]")
```

```
[10, 20, 30]
```

```
parse("[10]")
```

```
[10]
```

```
parse("[]")
```

```
[]
```

Abstract syntax trees

- Our current version of parser returns a Python list as a structured version of the input text
- This works well for this language because lists are a native data type in Python, but this is not the case in general
- In most cases, we want to construct an **Abstract Syntax Tree** (AST) as an intermediate representation during parsing
- An AST represents the essential structure of the input while abstracting away unnecessary syntactic details
- An AST is not the same as the *parse tree*
 - elements such as delimiters, separators, or auxiliary grammar symbols usually do not appear in the AST
- The AST is then used by later stages of the language processor, which may perform:
 - semantic analysis
 - program transformations
 - interpretation
 - code generation

✓  AST example: A language for lists

- Returning a Python list is already useful, but it does not explicitly represent the *syntactic structure* of the language
- For example, an AST for the list `[10, 20, 30]` could be:

```

List
└─ Cons
    └─ INT(10)
        └─ Cons
            └─ INT(20)
                └─ Cons
                    └─ INT(30)
                        └─ Nil

```

- In particular, this AST ignores elements such as:
 - delimiters like `[` and `]`
 - separators like `,`
 - auxiliary nonterminals introduced for parsing
 - this is distinct from the parse tree

```

List
└─ [
    └─ RestList
        └─ Elements
            └─ INT(10)
                └─ RestElements
                    └─ ,
                        └─ Elements
                            └─ INT(20)
                                └─ RestElements
                                    └─ ,
                                        └─ Elements
                                            └─ INT(30)
                                                └─ RestElements
                                                    └─ ε

```

- This AST can be represented using different data structures, for example:
 - simple nested tuples
 - classes representing AST nodes

✓ Tuple-based AST representation

- A simple way to represent an AST is using nested tuples
- For the list language we can define the following node types:

```
("List", elements)
("Cons", value, rest)
("Nil",)
```

- For example, the input text `[10,20,30]` becomes

```
("List",
  ("Cons", 10,
    ("Cons", 20,
      ("Cons", 30,
        ("Nil",))))))
```

- This representation directly reflects the recursive structure of the grammar
- No need to change the structure of the parser, we just need to return the proper AST node

```
def recognize_List():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "[":          # p1: List      → '[' RestList
        recognize_terminal("[")
        elems = recognize_RestList()
        return ("List",elems)      # the top-level list node
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'List':")

def recognize_RestList():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "INT":        # p2: RestList  → Elements ']'
        elems = recognize_Elements()
        recognize_terminal("]")
        return elems              # list with elements, return that node
    elif token_type == "]:":       # p3:           | ']'
        recognize_terminal("]")
        return ("Nil",)           # empty list, return nil node
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'RestLi")

def recognize_Elements():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "INT":        # p4: Elements  → INT RestElements
        elem = recognize_terminal("INT")
        elems = recognize_RestElements()
        return ("Cons", elem, elems) # cons node, element followed by the rest o
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'Elemen")
```

```
def recognize_RestElements():
    token_type, token_val, token_line, token_col = lookahead()
    if token_type == ",":
        # p5: RestElements → ',' Elements
        recognize_terminal(",")
        elems = recognize_Elements()
        return elems
    elif token_type == "]:
        # p6: | ε
        return ("Nil",)
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'RestEl")
```

```
def parse(text):
    lexer.input(text)
    next_token()
    result = recognize_List()
    if lookahead()[0] != "$":
        raise ParseError(f"Unexpected extra input: {lookahead()[0]}")
    return result
```

```
parse("[10, 20, 30]")
```

```
('List', ('Cons', 10, ('Cons', 20, ('Cons', 30, ('Nil',))))))
```

```
parse("[10]")
```

```
('List', ('Cons', 10, ('Nil',)))
```

```
parse("[]")
```

```
('List', ('Nil',))
```

✓ Class-based AST representation

- Tuple-based ASTs are simple and sufficient for small examples
- However, they are not suitable for complex languages and transformations
- For a proper representation, we should define a class hierarchy for the AST
- For the list language we can define three node types: `ListNode`, `ConsNode`, and `NilNode`
- Then, we just need to have the parser instantiate the proper AST objects

```
class ListNode:
    def __init__(self, elements):
        self.elements = elements

    def __repr__(self):
        return f"ListNode({self.elements})"
```



```

class ConsNode:
    def __init__(self, value, rest):
        self.value = value
        self.rest = rest

    def __repr__(self):
        return f"ConsNode({self.value}, {self.rest})"

class NilNode:
    def __repr__(self):
        return "NilNode()"

```

```

def recognize_List():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "[":
        # p1: List → '[' RestList
        recognize_terminal("[")
        elems = recognize_RestList()
        return ListNode(elems) # the top-level list node
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'List':")

def recognize_RestList():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "INT":
        # p2: RestList → Elements ']'
        elems = recognize_Elements()
        recognize_terminal("]")
        return elems # list with elements, return that node
    elif token_type == "]":
        # p3: | ']'
        recognize_terminal("]")
        return NilNode() # empty list, return nil node
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'RestLi")

def recognize_Elements():
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == "INT":
        # p4: Elements → INT RestElements
        elem = recognize_terminal("INT")
        elems = recognize_RestElements()
        return ConsNode(elem, elems) # cons node, element followed by the rest
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'Elemen")

def recognize_RestElements():
    token_type, token_val, token_line, token_col = lookahead()
    if token_type == ",":
        # p5: RestElements → ',' Elements
        recognize_terminal(",")
        elems = recognize_Elements()
        return elems # additional elements, return that node
    elif token_type == "]":
        # p6: | ε
        return NilNode() # no more elements, return nil node
    else:
        raise ParseError(f"Unexpected token when recognizing nonterminal 'RestEl")

```

```
def parse(text):
    lexer.input(text)
    next_token()
    result = recognize_List()      # store the parsing result
    if lookahead()[0] != "$":
        raise ParseError(f"Unexpected extra input: {lookahead()[0]}")
    return result
```

```
parse("[10, 20, 30]")
```

```
ListNode(ConsNode(10, ConsNode(20, ConsNode(30, NilNode()))))
```

```
parse("[10]")
```

```
ListNode(ConsNode(10, NilNode()))
```

```
parse("[]")
```

```
ListNode(NilNode())
```

✓ ✂ AST exercise: Simple arithmetic expressions

- Recall the following LL(1) CFG from the previous exercise:

```
p1: E      → T ERest
p2: ERest → '+' T ERest
p3:       | ε
p4: T      → F TRest
p5: TRest  → '*' F TRest
p6:       | ε
p7: F      → INT
```

- Write a top-down predictive parser to translate such expressions into an AST
- Then, evaluate the value of the expression by traversing the AST

```
import ply.lex as lex

# -----
# Lexer
# -----

class LexError(Exception):
    pass

tokens = ("INT",)
literals = "+*"

def t_INT(t):
```

```

        r"\d+"
        t.value = int(t.value)
        return t

t_ignore = " \t\n"

def t_error(t):
    raise LexError(f"Invalid symbol: {t.value[0]}")

__file__ = "Untitled.ipynb"    # sometimes needed by PLY in notebooks
lexer = lex.lex()

_lookahead = None

def next_token():
    global _lookahead
    _lookahead = lexer.token()

def lookahead():
    return / lookahead type, lookahead value, lookahead line no, lookahead lex

```

```

# -----
# Parser
# -----

class ParseError(Exception):
    pass

def recognize_terminal(expected_type):
    token_type, token_val, token_line, token_pos = lookahead()
    if token_type == expected_type:
        . . .

```